

栈与队列 (Stacks & Queues)

本文件夹包含 7 种栈和队列的实现，涵盖了顺序存储、链式存储以及各种变种。

📄 概览表

文件	数据结构	存储方式	特点	适用场景
SeqStack.cpp	顺序栈	动态数组	简洁高效	通用栈操作
LinkedStack.cpp	链式栈	链表	无大小限制	大规模栈操作
SeqQueue.cpp	顺序队列	循环数组	循环设计	通用队列操作
LinkedQueue.cpp	链式队列	链表	灵活易扩展	无限队列
SeqDeque.cpp	双端队列	循环数组	两端操作	双向处理
DualStack.cpp	双栈	共享数组	空间共享	空间受限
PQueue.cpp	优先队列	堆数组	按优先级	任务调度

📁 栈 (Stack) 详解

基本概念

- **LIFO** (Last In First Out) - 后进先出
- 只能从一端（栈顶）进行插入和删除
- 经典应用：函数调用栈、表达式求值、括号匹配

◇ SeqStack.cpp - 顺序栈

特点：

- 基于**动态数组**实现
- 简洁高效，缓存友好
- 大小固定，可能溢出

结构：

```
template <typename T>
class SeqStack {
private:
    T *elements;
    int top;           // 栈顶指针（指向下一个空位）
    int maxSize;
};
```

主要方法：

```
bool Push(const T &item)           // 入栈 O(1)
bool Pop(T &item)                   // 出栈 O(1)
bool getTop(T &item) const         // 取栈顶 O(1)
bool isEmpty() const              // 判空 O(1)
bool isFull() const               // 判满 O(1)
int getSize() const               // 获取大小 O(1)
void Clear()                      // 清空 O(1)
```

时间复杂度:

操作	复杂度
入栈	O(1)
出栈	O(1)
取栈顶	O(1)
查空/满	O(1)

使用示例:

```
SeqStack<int> stack(10);
stack.Push(1);
stack.Push(2);
stack.Push(3);

int top;
stack.getTop(top);           // top = 3
stack.Pop(top);              // top = 3, 栈顶出栈

cout << "Size: " << stack.getSize();
```

优点:

- ☒ 速度快
- ☒ 内存连续
- ☒ 缓存效率高

缺点:

- ☒ 大小固定
- ☒ 可能溢出

◇ LinkedStack.cpp - 链式栈

特点:

- 基于链表实现

- 大小动态，不会溢出
- 灵活但指针开销大

结构:

```
struct Linknode {
    T data;
    Linknode *next;
};

template <typename T>
class LinkedStack {
private:
    Linknode<T> *top;    // 指向栈顶
};
```

主要方法:

```
bool Push(const T &item)           // 入栈 O(1)
bool Pop(T &item)                   // 出栈 O(1)
bool getTop(T &item) const          // 取栈顶 O(1)
bool isEmpty() const                // 判空 O(1)
void MakeEmpty()                   // 清空 O(n)
int getSize()                       // 获取大小 O(n)
```

时间复杂度:

操作	复杂度
入栈	O(1)
出栈	O(1)
取栈顶	O(1)
获取大小	O(n)

使用示例:

```
LinkedStack<int> stack;
stack.Push(10);
stack.Push(20);
stack.Push(30);

int top;
stack.getTop(top);    // top = 30
stack.Pop(top);       // 删除30

cout << "Is empty: " << (stack.isEmpty() ? "Yes" : "No");
```

优点:

- ☒ 无大小限制
- ☒ 动态扩展
- ☒ 不会溢出

缺点:

- ✗ 指针开销
- ✗ 缓存不友好
- ✗ 获取大小慢

队列 (Queue) 详解

基本概念

- **FIFO** (First In First Out) - 先进先出
- 只能从队尾插入, 从队头删除
- 经典应用: 消息队列、BFS、操作系统进程调度

◇ SeqQueue.cpp - 顺序队列

特点:

- 基于**循环数组**实现
- 防止虚假满
- 采用取模运算循环使用数组

结构:

```
template <typename T>
class SeqQueue {
private:
    T *data;           // 数据数组
    int front;         // 队头指针
    int rear;          // 队尾指针
    int maxSize;
};
```

循环数组原理:

初始状态:
front=0, rear=0, maxSize=5
[_, _, _, _, _]

入队 1,2,3:
front=0, rear=3

```
[1, 2, 3, _, _]
```

出队 1,2:

```
front=2, rear=3
```

```
[X, X, 3, _, _]
```

继续入队 4,5:

```
front=2, rear=0 (循环)
```

```
[5, 2, 3, 4, X]
```

主要方法:

```
bool Enqueue(const T &item)    // 入队 O(1)
bool Dequeue(T &item)          // 出队 O(1)
bool getFront(T &item) const   // 取队头 O(1)
bool isEmpty() const           // 判空 O(1)
bool isFull() const            // 判满 O(1)
int getSize() const            // 获取大小 O(1)
void Clear()                   // 清空 O(1)
```

判空/判满的巧妙设计:

```
// 判空: front == rear
bool isEmpty() const {
    return front == rear;
}

// 判满: (rear + 1) % maxSize == front
// 空出一个位置来区分空和满
bool isFull() const {
    return (rear + 1) % maxSize == front;
}
```

使用示例:

```
SeqQueue<int> queue(10);
queue.Enqueue(1);
queue.Enqueue(2);
queue.Enqueue(3);

int front;
queue.getFront(front); // front = 1
queue.Dequeue(front);  // 删除1

cout << "Size: " << queue.getSize();
```

时间复杂度：

操作	复杂度
入队	O(1)
出队	O(1)
取队头	O(1)
判空/满	O(1)

优点：

- ☒ 速度快
- ☒ 空间利用率高（循环设计）
- ☒ 简单易用

缺点：

- ☒ 大小固定
- ☒ 可能溢出

◇ LinkedQueue.cpp - 链式队列

特点：

- 基于**链表**实现
- 大小无限，灵活扩展
- 不需要判满

结构：

```
struct ListNode {
    T data;
    ListNode *next;
};

template <typename T>
class LinkedQueue {
private:
    ListNode<T> *front;    // 队头
    ListNode<T> *rear;    // 队尾
};
```

主要方法：

```
bool Enqueue(const T &item)    // 入队 O(1)
bool Dequeue(T &item)          // 出队 O(1)
bool getFront(T &item) const   // 取队头 O(1)
```

```
bool isEmpty() const           // 判空 O(1)
void MakeEmpty()              // 清空 O(n)
```

入队操作 (尾插法):

```
初始: front=null, rear=null

入队1: front=[1]->null, rear=[1]->null

入队2: front=[1]->[2]->null, rear=[2]->null

入队3: front=[1]->[2]->[3]->null, rear=[3]->null
```

使用示例:

```
LinkedList<int> queue;
queue.Enqueue(10);
queue.Enqueue(20);
queue.Enqueue(30);

int val;
queue.GetFront(val);    // val = 10
queue.Dequeue(val);     // 删除10, val = 10
```

优点:

- ☒ 无大小限制
- ☒ 动态扩展
- ☒ 不会溢出

缺点:

- ☒ 指针开销
- ☒ 缓存不友好

特殊数据结构

◇ SeqDeque.cpp - 双端队列 (Deque)

特点:

- **继承自 SeqQueue**
- 两端都可以进行插入和删除
- 最灵活的队列

额外方法:

```
// 继承的标准队列操作
bool Enqueue(const T &item)    // 队尾入队
bool Dequeue(T &item)          // 队头出队

// 双端特殊操作
bool EnQueueHead(const T &item) // 队头入队 O(1)
bool DeQueueTail(T &item)       // 队尾出队 O(1)
bool getHead(T &item) const     // 获取队头 O(1)
bool getTail(T &item) const     // 获取队尾 O(1)
```

使用示例:

```
SeqDeque<int> deque(10);

// 两端入队
deque.EnqueueTail(1); // 队尾入队
deque.EnqueueHead(0); // 队头入队
deque.EnqueueTail(2); // -> [0, 1, 2]

// 两端出队
int val;
deque.DeQueueHead(val); // val = 0
deque.DeQueueTail(val); // val = 2 -> [1]
```

应用场景:

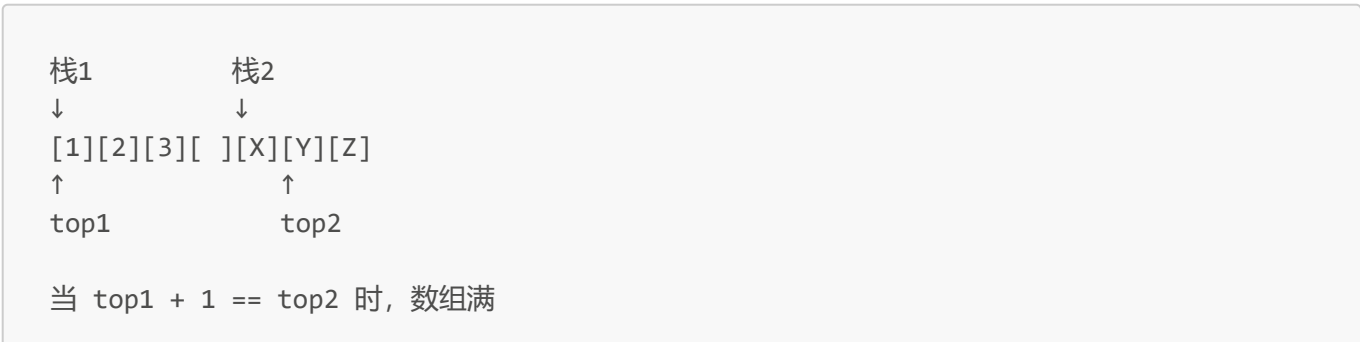
- 滑动窗口最大值
- 双向广度优先搜索
- 机器人的双向运动

◇ DualStack.cpp - 双栈

特点:

- 两个栈共享一个数组
- 从两端向中间增长
- 空间利用率最高

结构图:



实现原理:

```
template <typename T>
class DualStack {
private:
    T *elements;
    int top1;      // 栈1顶部 (从0向上)
    int top2;      // 栈2顶部 (从maxSize-1向下)
    int maxSize;
};
```

主要方法:

```
bool Push1(const T &item)    // 栈1入栈
bool Push2(const T &item)    // 栈2入栈
bool Pop1(T &item)           // 栈1出栈
bool Pop2(T &item)           // 栈2出栈
bool getTop1(T &item)        // 取栈1顶
bool getTop2(T &item)        // 取栈2顶
bool isEmpty1()              // 栈1是否为空
bool isEmpty2()              // 栈2是否为空
```

使用示例:

```
DualStack<int> ds(100);

// 栈1: 1, 2, 3
ds.Push1(1);
ds.Push1(2);
ds.Push1(3);

// 栈2: X, Y, Z
ds.Push2(100);
ds.Push2(200);
ds.Push2(300);

int val;
ds.getTop1(val);    // val = 3
ds.getTop2(val);    // val = 300
```

优点:

- ☒ 空间利用率最高
- ☒ 动态平衡
- ☒ 固定大小

缺点:

- ✕ 较复杂
- ✕ 只能两个栈

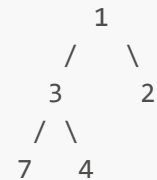
◇ PQueue.cpp - 优先队列 (Priority Queue)

特点:

- 基于**最小堆**实现
- 按优先级（值大小）出队，不是FIFO
- 自动排序

堆的性质:

最小堆: 父 $\leq$ 左子 且 父 $\leq$ 右子



满足: $1 \leq 3 \leq 7$, $1 \leq 3 \leq 4$, $1 \leq 2$

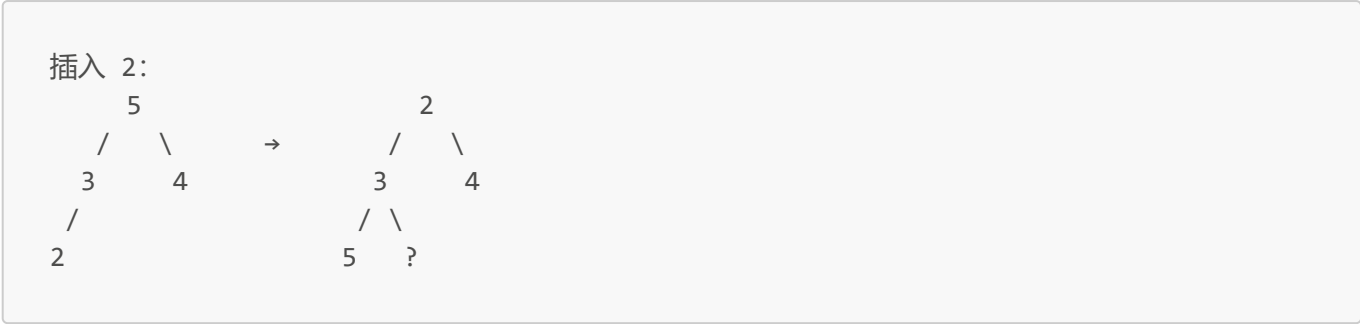
结构:

```
template <typename T>
class PQueue {
private:
    T *pqelems;    // 堆数组
    int count;     // 元素个数
    int maxSize;
    void adjust(); // 堆化操作
};
```

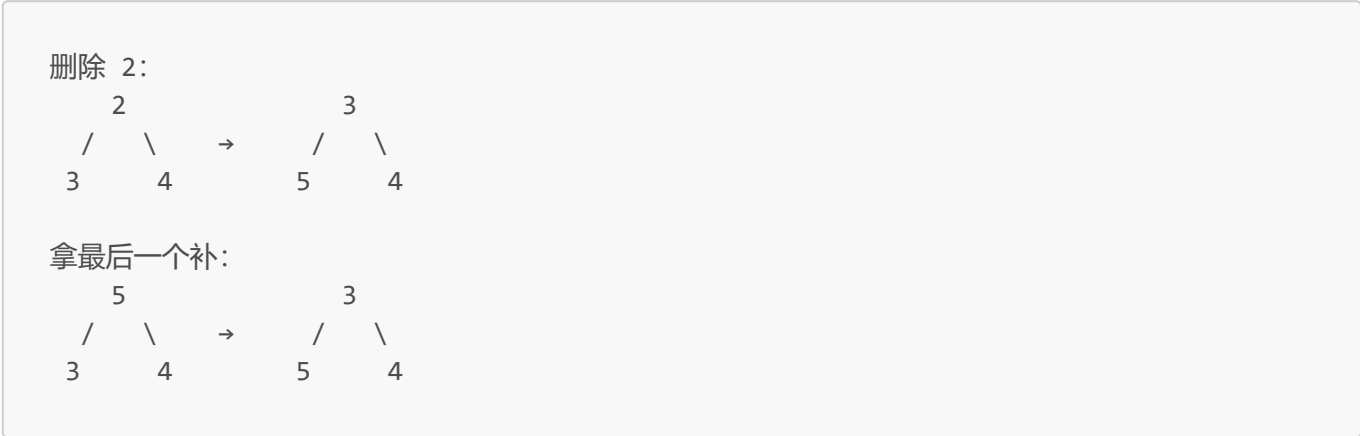
主要方法:

```
bool Insert(const T &elem)    // 插入元素 O(log n)
bool RemoveMin(T &elem)       // 移除最小 O(1)
bool getMin(T &elem) const    // 获取最小 O(1)
bool IsEmpty() const          // 判空 O(1)
bool IsFull() const           // 判满 O(1)
int getSize() const           // 获取大小 O(1)
```

插入过程 (向上调整) :



删除最小元素（向下调整）：



使用示例：

```
PQueue<int> pq(10);

pq.Insert(5);    // 插入
pq.Insert(3);
pq.Insert(8);
pq.Insert(1);

int min;
pq.getMin(min);  // min = 1 (最小)
pq.RemoveMin(min); // 删除1

pq.getMin(min);  // min = 3 (新最小)
```

时间复杂度：

操作	复杂度
插入	O(log n)
删除最小	O(log n)
获取最小	O(1)

应用场景：

- 任务调度（高优先级先执行）

- Dijkstra 算法
- 哈夫曼编码
- 医院挂号系统

性能对比

操作	SeqStack	LinkedStack	SeqQueue	LinkedQueue	SeqDeque	DualStack	PQueue
插入	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(logn)
删除	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(logn)
查询	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)	O(1)
内存	固定	动态	固定	动态	固定	固定	固定
溢出	☑	✗	☑	✗	☑	☑	☑

编译与运行

```
# 编译所有文件
g++ -Wall -Wextra -g3 SeqStack.cpp -o output/SeqStack.exe
g++ -Wall -Wextra -g3 LinkedStack.cpp -o output/LinkedStack.exe
g++ -Wall -Wextra -g3 SeqQueue.cpp -o output/SeqQueue.exe
g++ -Wall -Wextra -g3 LinkedQueue.cpp -o output/LinkedQueue.exe
g++ -Wall -Wextra -g3 SeqDeque.cpp -o output/SeqDeque.exe
g++ -Wall -Wextra -g3 DualStack.cpp -o output/DualStack.exe
g++ -Wall -Wextra -g3 PQueue.cpp -o output/PQueue.exe

# 运行
cd output
SeqStack.exe
SeqQueue.exe
# 等等
```

选择建议

选择栈

需求	推荐	原因
通常情况	SeqStack	简单高效
大规模	LinkedStack	无大小限制
空间受限	SeqStack	最小开销

选择队列

需求	推荐	原因
通常情况	SeqQueue	简单高效
大规模	LinkedQueue	无大小限制
两端操作	SeqDeque	双向灵活
优先级处理	PQueue	自动排序
极端空间限制	DualStack	利用率最高

 经典应用

1. 括号匹配 (使用栈)

检查 "{[()]}" 是否匹配
用栈存储左括号，遇到右括号时弹出匹配

2. 表达式求值 (使用栈)

中缀 → 后缀表达式 → 求值
"3 + 4 * 2 / (1 - 5)"
= "3 4 2 * 1 5 - / +"
= 3 + (8 / (-4))

3. 函数调用栈 (使用栈)

main() 调用 f1(), f1() 调用 f2()
栈: [main] → [main, f1] → [main, f1, f2]
返回时弹栈

4. BFS 广度优先搜索 (使用队列)

从起点开始，逐层扩展
队列存储当前层的所有节点

5. 操作系统进程调度 (使用队列)

进程按入队顺序执行
或用优先队列按优先级执行

🔍 常见问题

Q: 什么时候用 `LinkedList`? A: 当队列元素个数不确定, 且可能很大时。

Q: `PQueue` 什么时候用? A: 需要按优先级处理任务, 如医院挂号、任务调度。

Q: `DualStack` 太复杂, 有必要吗? A: 在嵌入式系统或空间极其受限时有用。大多数情况不需要。

Q: `SeqQueue` 为什么要用循环? A: 避免虚假满, 提高空间利用率。

📖 学习路线

1. 第一阶段 - 基础

- 理解 LIFO/FIFO 概念
- 学习 `SeqStack` 和 `SeqQueue`

2. 第二阶段 - 进阶

- 学习 `LinkedList` 和 `LinkedList`
- 理解链表实现

3. 第三阶段 - 高级

- 学习 `SeqDeque` 和 `DualStack`
- 理解特殊设计

4. 第四阶段 - 优化

- 学习 `PQueue`
 - 理解堆结构和优先级
-

最后更新: 2026年1月18日 作者: 数据结构学习项目